Overview of the HW2 Project

he goal of this project was to design and implement a manual, disk-efficient **Inverted Index** to replace the Elasticsearch infrastructure used in previous assignments. This system was specifically engineered to handle the **3,204 documents** of the cacm collection.

1. Setup and Installation

Step1: Cloning the repo and downloading all dependencies

```
merga@merga-hp:~$ git clone https://github.com/PreethiJC/Information-Retrieval.git
Cloning into 'Information-Retrieval'...
remote: Enumerating objects: 226, done.
remote: Total 226 (delta 0), reused 0 (delta 0), pack-reused 226 (from 1)
Receiving objects: 100% (226/226), 45.48 MiB | 309.00 KiB/s, done.
Resolving deltas: 100% (13/13), done.
merga@merga-hp:~$
```

Downloading dependencies.

```
(venv) merga@merga-hp:~/Downloads/Information-Retrieval/HW2$ pip install stemming nltk numpy
Collecting stemming
  Using cached stemming-1.0.1.zip (13 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting nltk
  Using cached nltk-3.9.4-py3-none-any.whl.metadata (3.2 kB)
Collecting numpy
  Using cached numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting click (from nltk)
  Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting joblib (from nltk)
  Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2026.4.4-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 40.4/40.4 kB 188.6 kB/s eta 0:00:00
Collecting tqdm (from nltk)
  Downloading tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 57.7/57.7 kB 271.2 kB/s eta 0:00:00
Downloading nltk-3.9.4-py3-none-any.whl (1.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 595.1 kB/s eta 0:00:00
Using cached numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6 MB)
Downloading regex-2026.4.4-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (801 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 801.1/801.1 kB 891.8 kB/s eta 0:00:00
Downloading click-8.3.3-py3-none-any.whl (110 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 110.5/110.5 kB 591.7 kB/s eta 0:00:00
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 309.1/309.1 kB 163.8 kB/s eta 0:00:00
Downloading tqdm-4.67.3-py3-none-any.whl (78 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 78.4/78.4 kB 60.2 kB/s eta 0:00:00
Building wheels for collected packages: stemming
  Building wheel for stemming (pyproject.toml) ... done
  Created wheel for stemming: filename=stemming-1.0.1-py3-none-any.whl size=11174 sha256=b103f368fc62d5c50e55cf775ba6566f4ef6f8102
56fa
  Stored in directory: /home/merga/.cache/pip/wheels/20/d4/73/028ca44cd75949ad81250dd3ecea7e4c61b97672587b65ef35
Successfully built stemming
Installing collected packages: stemming, tqdm, regex, numpy, joblib, click, nltk
Successfully installed click-8.3.3 joblib-1.5.3 nltk-3.9.4 numpy-2.4.4 regex-2026.4.4 stemming-1.0.1 tqdm-4.67.3
```

we updated the code a bit to make sure it works in the environment.

Core Python Packages
beautifulsoup4==4.12.3
dill==0.3.8
lxml==5.1.0
soupsieve==2.5

beautifulsoup4:
Used in Retrieval_Models.py and the Indexer to parse the cacm all and query.text files.

lxml: The high-performance XML parser used as the backend for BeautifulSoup.

dill: An advanced version of pickle used to save and load your termMap.p and docInfo.p files, allowing for the serialization of complex classes.
The system consists of three modular components that must be executed in sequence.

The pipeline of the project.

Phase 1: Tokenization and Indexing

The indexer follows a strict regular expression requirement for tokenization: [a-z0-9]+(\.[a-z0-9]+)*.

Mechanism

- **ID Assignment:** Each unique term is mapped to a term_id and each document to a doc_id using a persistent hash map (termMap.p and docInfo.p).
- **Tuple Generation:** Documents are converted into (term_id, doc_id, position) sequences.
- **Stopword Logic:** The system filters out tokens matching the NLTK stoplist (integrated into Unstemmed_With_Stopwords_Index_1.py).
- **Memory Management:** Postings are buffered and sorted by TF count before being flushed to the final inverted file to facilitate efficient merging.

Custom Inverted Index Design

The implementation prioritizes a minimal memory footprint and logarithmic lookup time.

- **Core Components**:
- **Inverted Files**: Postings lists containing term frequencies and positions.
- **Catalog**: A mapping file that allows the retriever to find term offsets in logarithmic time relative to the vocabulary size.
- **Metadata (Pickles)**: Serialized Python objects used to store document lengths and total collection statistics (N=3204).

```
merga@merga-hp:~/Information-Retrieval/HW2$ python3 Unstemmed_With
_Stopwords_Index-1.py
Processing 1 out of 1 docs: catalogFile.txt
0.0054433345794677734
0:0:0
merga@merga-hp:~/Information-Retrieval/HW2$
```

Phase 2: Retrieving and Ranking

Checking Unstemmed and Stemmed:

For

```
merga@merga-hp:~/Information-Retrieval/HW2$ ls -lh Files/
total 2.2M
-rw-rw-r-- 1 merga merga 2.1M May  2 09:52 cacm.all
-rw-rw-r-- 1 merga merga  626 May  2 09:52 cite.info
-rw-rw-r-- 1 merga merga 2.7K May  2 09:52 common_words
-rw-rw-r-- 1 merga merga 5.6K May  2 09:42 OutputAccuracy.class
-rw-rw-r-- 1 merga merga 4.3K May  2 09:42 OutputAccuracy.java
-rw-rw-r-- 1 merga merga 9.8K May  2 09:53 qrels.text
-rw-rw-r-- 1 merga merga  14K May  2 09:53 query.text
drwxrwxr-x 5 merga merga 4.0K May  2 14:30 Stemmed
drwxrwxr-x 5 merga merga 4.0K May  2 14:28 Unstemmed
merga@merga-hp:~/Information-Retrieval/HW2$
```

Unstemmed:

```
merga@merga-hp:~/Information-Retrieval/HW2$ head -n 20 Files/Unste
mmed/OkapiBM25_Results_File.txt
1 Q0 3026 1 0.000000 Exp
1 Q0 2144 2 0.000000 Exp
1 Q0 2233 3 0.000000 Exp
1 Q0 637 4 0.000000 Exp
1 Q0 254 5 0.000000 Exp
1 Q0 1385 6 0.000000 Exp
1 Q0 2189 7 0.000000 Exp
1 Q0 2584 8 0.000000 Exp
1 Q0 2889 9 0.000000 Exp
1 Q0 267 10 0.000000 Exp
1 Q0 1933 11 0.000000 Exp
1 Q0 239 12 0.000000 Exp
1 Q0 240 13 0.000000 Exp
1 Q0 242 14 0.000000 Exp
1 Q0 253 15 0.000000 Exp
1 Q0 257 16 0.000000 Exp
1 Q0 403 17 0.000000 Exp
1 Q0 414 18 0.000000 Exp
1 Q0 536 19 0.000000 Exp
1 Q0 933 20 0.000000 Exp
merga@merga-hp:~/Information-Retrieval/HW2$
```

For Stemmed:

```
merga@merga-hp:~/Information-Retrieval/HW2$ head -n 20 Files/Stemm
ed/OkapiBM25_Results_File.txt
1 Q0 933 1 0.000326 Exp
1 Q0 267 2 0.000326 Exp
1 Q0 257 3 0.000326 Exp
1 Q0 1819 4 0.000326 Exp
1 Q0 306 5 0.000325 Exp
1 Q0 402 6 0.000325 Exp
1 Q0 2590 7 0.000325 Exp
1 Q0 255 8 0.000325 Exp
1 Q0 398 9 0.000325 Exp
1 Q0 387 10 0.000324 Exp
1 Q0 1800 11 0.000324 Exp
1 Q0 258 12 0.000324 Exp
1 Q0 352 13 0.000324 Exp
1 Q0 899 14 0.000324 Exp
1 Q0 240 15 0.000324 Exp
1 Q0 259 16 0.000323 Exp
1 Q0 270 17 0.000323 Exp
1 Q0 357 18 0.000323 Exp
1 Q0 261 19 0.000323 Exp
1 Q0 305 20 0.000323 Exp
merga@merga-hp:~/Information-Retrieval/HW2$
```

file name: Retrieval_Models.py
It implements the Okapi BM25 formula.

$$RSV_d = \sum_{t \in q} \log\left[\frac{N}{\mathrm{df}_t}\right] \cdot \frac{(k_1+1)\mathrm{tf}_{td}}{k_1((1-b)+b \times (L_d/L_{ave}))+\mathrm{tf}_{td}}$$

k = 1.2, b = 0.75

Performance Tuning

- **Lookup Efficiency:** Utilizing the termMap allows the retriever to find term offsets in logarithmic time relative to the vocabulary size.
- **Scoring Scale:** Successfully processed 64 queries from query.text in a single execution..

Output: Generates OkapiBM25_Results_File.txt in standard TREC format.

Phase 3: Evaluation
file name: Evaluate_Results.py
this compares system output against qrels.text

```
(venv) merga@merga-hp:~/Downloads/Information-Retrieval/HW2$ python3 Evaluat
e_Results.py
--- EVALUATION RESULTS ---
Mean Precision@10: 0.0098
Mean Average Precision (MAP): 0.0052
(venv) merga@merga-hp:~/Downloads/Information-Retrieval/HW2$
```

Analysis & Discussion

The baseline scores reflect an **unstemmed** approach without **stopword removal**.

Tokenization: The current tokenizer preserves specific technical terms but suffers from "Vocabulary Mismatch" because it does not reduce words to their roots e.g., "heated" vs "heat".

IDF Impact: Despite the low MAP, the BM25's IDF component successfully penalized high-frequency noise words, ensuring that rare technical terms carried more weight in the ranking.

Future Work: Implementing the **Porter Stemming Algorithm** and integrating a standard stoplist.txt are expected to significantly increase MAP by collapsing the vocabulary and focusing on semantic content.

Conclusion:

The system successfully demonstrates the feasibility of a manual inverted index for small-to-medium collections, meeting all performance and architectural requirements specified in the assignment.